

ShiftSwap: A Shift Schedule and Substitute Management System for Small Part-Time Workplaces

Requirements specification and analysis models

Table 1. Team 7

Name	Student ID	Affiliation	Role
Hyewon Kim (김혜원)	24102028	Dept. of Industrial Engineering, ITM Program, SeoulTech	Requirements lead
Hyoungdo Kim (김형도)	18102075	Dept. of Industrial Engineering, ITM Program, SeoulTech	Project manager and design/implementation lead
Minkyung Kim (김민경)	23102004	Dept. of Industrial Engineering, ITM Program, SeoulTech	UI and quality lead

Course: Software Engineering, Seoul National University of Science and Technology (SeoulTech)

Submitted to: jihjim@gmail.com

Date: 16 October 2026

Contents

- 1. Introduction
- 2. Problem description
 - 2.1 Background and current process
 - 2.2 Problems P1–P3
 - 2.3 Proposed solution and system environment
 - 2.4 Actors
- 3. Requirements elicitation
- 4. Functional requirements specification
 - 4.1 Functional requirements
 - 4.2 Business rules
 - 4.3 Use-case descriptions
- 5. Non-functional requirements specification
- 6. Scenarios
- 7. UML diagrams for requirements analysis
 - 7.1 Use case diagram
 - 7.2 Activity diagrams
 - 7.3 State machine diagrams
 - 7.4 Analysis class diagram
- 8. Main user interface screens
- 9. Requirements validation
- 10. Progress and next steps
- References

1. Introduction

This interim report specifies the requirements of ShiftSwap and presents the UML models used to analyse them. ShiftSwap is a web application for a small store with part-time workers. It manages the weekly shift schedule, finds substitutes when a worker cannot work a shift, and computes weekly holiday allowance and monthly pay under Korean labor law.

The proposal (Week 5) defined the problem, the goals, the nine main functions and the project plan. This report refines them: problem (Section 2), elicitation (3), functional and non-functional requirements (4, 5), scenarios (6), analysis models (7), main screens (8) and validation (9). The team follows the incremental model with prototyping of the user interface (Chapter 1): increment 1 delivers the *Must* use cases and increment 2 the *Should* use cases. The screens in Section 8 come from the prototype, which we use to validate requirements with the owner and workers before the design is fixed. All identifiers (UC, FR, NFR, BR, TC) come from one project specification that the team uses as the single source of truth; the same identifiers appear in the models, the code and the tests.

2. Problem description

2.1 Background and current process

In small stores (cafés, convenience stores, restaurants) with fewer than ten part-time workers, each worker has fixed weekly shifts, for example every Monday 18:00–23:00. When a worker cannot come, the owner phones or messages other workers one by one to find a substitute. The owner then edits a spreadsheet by hand. At month end the owner must re-check each worker's weekly hours to decide whether the weekly holiday allowance (주휴수당) is owed.

Our reference store is the workplace of team member Hyewon Kim. The owner also acts as the manager. There, finding a substitute typically takes 3 to 5 contacts per request. The current process has four manual steps: (1) the worker informs the owner; (2) the owner contacts other workers one at a time; (3) the owner edits the spreadsheet and informs both workers; (4) at month end the owner re-counts hours and computes pay. None of these steps is recorded except the final spreadsheet.

Korean law adds rules that depend on these hours. A worker whose weekly contractual hours are 15 or more and who worked every scheduled day of the week is entitled to a paid weekly holiday (Labor Standards Act Art. 55 ①, Art. 18 ③, Enforcement Decree Art. 30 ①) [3, 5]. The 2026 minimum hourly wage is KRW 10,320 [1, 2]. Overtime and night premiums apply only in workplaces with five or more regular employees (Art. 11, 56) [3]. The minimum wage may be reduced to 90 % during probation only if several conditions hold (Minimum Wage Act Art. 5 ②, Enforcement Decree Art. 3) [4].

2.2 Problems P1–P3

P1 Finding a substitute costs the owner repeated one-to-one contacts (reference store: typically 3–5 contacts per request).

P2 The schedule, weekly hours and holiday-allowance eligibility of *two* workers change with every swap, and all recalculation is manual.

P3 Legal pay rules (minimum wage, holiday allowance, 5-employee threshold for premiums, probation reduction) are applied from memory, so errors are not caught before payday.

Existing shift-scheduling products (Shiftee, the work-management features of Albamon and Alba Cheonguk, When I Work, Homebase, Deputy) manage schedules and shift trades. None of those we examined links substitute approval to Korean holiday-allowance eligibility at the moment of approval [10].

2.3 Proposed solution and system environment

ShiftSwap replaces steps (2) to (4). A worker creates a substitute request for one of their shifts. The system selects every eligible colleague and puts the request in their inbox at once. The first colleague to accept becomes the acceptor. The owner sees what the swap does to both workers' hours and holiday-allowance eligibility, and approves or rejects it. On approval the schedule and weekly summaries are updated. At month end the system produces a payroll draft and flags minimum-wage problems.

The system runs entirely in a web browser. It is a static web application (HTML, CSS and JavaScript modules) with an embedded SQLite database (sql.js, compiled to WebAssembly) [9]. Data is saved in the browser's local storage after every transaction. There is no server and no installation. The application is published on GitHub Pages. The same device can be used by the owner on a laptop and by workers on phones, because every screen works at 360 px and 1440 px width. Because the application is a demonstration system without a server, sign-in is done by choosing a demo account, and

the passage of time is simulated by an adjustable demo clock. The default demo time is Monday 2026-09-28 09:00.

The demo data describes Dalbit Café (달빛카페), a fictional store with four regular employees and the default substitution-attendance policy EXCUSED. It is used in all scenarios and screenshots of this report.

Table 2. Demo store: owner and workers

ID	Name	Wage (KRW/h)	Contract	Fixed shifts (h/week)	Availability
1	Park Jiyoung (owner)	–	–	–	–
2	Lee Seoyeon	10,500	2026-03-02 to 2027-02-28	Mon, Wed 18:00–23:00 (10)	Tue 17:00–23:00; Thu 17:00–23:00; Sat 10:00–22:00
3	Choi Minho	10,320	2026-06-01 to 2026-12-31	Tue, Thu 18:00–23:00; Sat 12:00–18:00 (16)	Mon 17:00–23:00; Wed 17:00–23:00
4	Jung Hana	10,320	2026-09-01, open-ended; probation to 2026-11-30; not simple labor	Fri 17:00–23:00; Sun 10:00–16:00 (12)	Mon 18:00–23:00; Sat 10:00–18:00
5	Kang Doyun	10,000	2026-08-15 to 2026-11-15; simple labor	Sat 10:00–16:00; Sun 16:00–22:00 (12)	Wed 18:00–23:00; Thu 18:00–23:00; Fri 17:00–23:00

The minimum wage table holds 2025: KRW 10,030 and 2026: KRW 10,320. The shifts of the weeks starting 2026-09-21 and 2026-09-28 are generated, and all shifts of the week of 2026-09-21 have confirmed attendance. One open request exists: Lee Seoyeon's request for Wednesday 2026-09-30 18:00–23:00 (Section 6, S1).

2.4 Actors

The system has three actors. **A1 Worker** (primary, human) is a part-time employee who views their own schedule, registers availability, requests substitutes, responds to requests and records attendance. **A2 Owner** (primary, human) is the store owner, who also acts as manager: the owner registers workers and fixed schedules, approves or rejects substitutions, confirms attendance, reviews weekly summaries, confirms monthly payroll and edits settings. **A3 System Clock** (secondary, system) is a time-triggered actor. It expires unanswered or unapproved requests at their deadline and closes weeks for the weekly summary; in the demo application it is driven by the adjustable demo clock.

3. Requirements elicitation

Chapter 8 describes requirements elicitation as work with the stakeholders to find out about the application domain, the services the system should provide and its constraints. Our stakeholders are the

store owner, the part-time workers and, indirectly, the labor law that fixes the pay rules. We use four sources.

1. **Domain experience.** Hyewon Kim works part-time in the reference store. Her experience gave the problem statement and the figure of typically 3–5 contacts per substitute request.
2. **Interviews.** Semi-structured interviews with the owner of the reference store and two of Hyewon's co-workers. Hyewon Kim leads each interview and a second member takes notes. Each interview is planned for about 30 minutes.
3. **Document analysis.** The owner's schedule spreadsheet (structure only; no personal data is copied) and the legal sources on minimum wage, weekly holiday and probation [1–5].
4. **Prototype sessions.** After the first interviews the owner and the co-workers are shown the UI prototype (Section 8) and asked to perform one task each.

Table 3. Interview plan

Owner of the reference store — current process, decisions the owner makes, pay calculation
1. How do you learn that a worker cannot come? How far in advance? 2. Whom do you contact first, and how do you decide? How many contacts does one request take? 3. What do you check before you agree to a swap? 4. Where do you record the change? Who else needs to know? 5. How do you decide at month end whether weekly holiday allowance is owed? 6. If a worker hands over a shift by agreement, do you count that week as full attendance? 7. How many regular employees does the store have? Do you pay night or overtime premiums? 8. Do any workers have a probation period or a reduced wage? 9. Which step of the current process costs you the most time? 10. What should the system never do without your approval?
Two co-workers — the worker's view of requesting and taking substitute shifts
1. How often do you need a substitute? For what reasons? 2. How do you find out that a colleague needs a substitute? 3. What makes you accept or decline a request? 4. Has it happened that two people agreed to take the same shift? 5. When would you like the request to stop being open (deadline)? 6. Would you enter your available times in advance? At what level of detail? 7. Which information about you should colleagues not see? 8. How do you check your own hours and pay today?

The questions were derived from P1–P3 and from the open points in the business rules. Question 6 to the owner addresses the only rule where the law leaves room for interpretation (BR-06). Question 4 to the workers addresses the concurrency requirement (BR-02, NFR-04). Question 7 to the workers addresses privacy (NFR-11). The notes of each interview are kept by the requirements lead. This report does not include interview results beyond the problem statement; answers that change a requirement will be recorded in the final report together with the changed requirement.

4. Functional requirements specification

4.1 Functional requirements

Table 4 lists the 21 functional requirements. Each requirement is traced to at least one use case. The priority is that of its use case: *Must* requirements are delivered in increment 1 and *Should* requirements in increment 2.

Table 4. Functional requirements

ID	Requirement	UC	Priority
FR-01	The owner shall register a worker with name, phone, hourly wage (KRW), contract start date, contract end date, probation end date (optional), simple-labor flag, and active flag.	UC-01	Must
FR-02	The owner shall register one or more fixed weekly shifts per worker (weekday, start time, end time); fixed shifts of the same worker must not overlap.	UC-01	Must
FR-03	A worker shall register availability slots (weekday, start, end) during which they can take substitute shifts.	UC-02	Must
FR-04	The system shall generate the shifts of a given week (Mon–Sun) from all active fixed schedules; generating the same week twice shall not create duplicates.	UC-03	Must
FR-05	The owner shall be able to add, move or delete a single shift of a generated week.	UC-03	Must
FR-06	A worker shall create a substitute request for one of their own future SCHEDULED shifts, with an optional reason and a response deadline.	UC-04	Must
FR-07	On creation the system shall compute the eligible candidates (BR-01), create one SubRequestTarget per candidate, and put a notification in each candidate's inbox at once.	UC-04	Must
FR-08	If no worker is eligible, the system shall still create the request and notify the owner that no candidate exists.	UC-04	Must
FR-09	A candidate shall accept or decline a request from their inbox. The first acceptance makes that candidate the request's acceptor (BR-02); later acceptances shall be refused with an explanatory message.	UC-05	Must
FR-10	When a request is accepted, the system shall notify the requester and the owner, and close all other pending targets.	UC-05	Must
FR-11	The approval screen shall show, for both the requester and the acceptor, the weekly contractual hours, weekly scheduled hours and holiday-allowance eligibility before and after the swap, and highlight any eligibility change.	UC-06	Must
FR-12	On approval the system shall reassign the shift to the acceptor (keeping the original worker), recompute the weekly summaries of both workers, and notify both. On rejection the shift stays unchanged and both are notified.	UC-06	Must
FR-13	A worker shall record actual start and end time of a shift they worked; the owner shall confirm it or mark the shift ABSENT.	UC-07	Should

ID	Requirement	UC	Priority
FR-14	The system shall compute, per worker and week, contractual hours, scheduled hours, actual hours, perfect-attendance flag, holiday-allowance eligibility and holiday-allowance hours (BR-06, BR-07).	UC-08	Must
FR-15	The owner shall generate a monthly payroll draft per worker (base pay, holiday allowance, premium pay, total) and confirm it; a confirmed payroll is read-only.	UC-09	Should
FR-16	The payroll screen shall warn when a worker's hourly wage is below the applicable minimum wage (BR-09), and apply the probation reduction only when BR-10 holds.	UC-09	Should
FR-17	The owner shall edit workplace settings: store name, number of regular employees, substitution-attendance policy (BR-06), and yearly minimum wage rows.	UC-10	Should
FR-18	When a request's deadline passes, or its shift starts, while it is still REQUESTED or ACCEPTED, the system shall mark it EXPIRED, close its targets and notify requester and owner.	UC-11	Must
FR-19	A requester shall cancel their own request while it is REQUESTED or ACCEPTED; all targets are closed and notified.	UC-12	Should
FR-20	Users shall sign in by choosing a demo account; the UI shows only the menus of that role.	UC-13	Must
FR-21	Every user shall have an in-app notification inbox listing notifications newest first with unread count.	UC-04-06, 11, 12	Must

4.2 Business rules

The business rules are the domain requirements (Chapter 8) that several functional requirements share. They are stated once here and referenced by ID.

Table 5. Business rules

ID	Rule
BR-01	Eligible candidates. A worker <i>W</i> is eligible for a request on shift <i>S</i> (date <i>d</i> , start <i>s</i> , end <i>e</i>) iff: <i>W</i> is active; <i>W</i> ≠ requester; <i>W</i> has no shift on date <i>d</i> overlapping [<i>s</i> , <i>e</i>); <i>W</i> has an availability slot on weekday(<i>d</i>) with slotStart ≤ <i>s</i> and slotEnd ≥ <i>e</i> .
BR-02	First acceptance wins. Acceptance succeeds only if the request is REQUESTED at the moment of the transaction; it sets request.status = ACCEPTED, request.acceptorId = <i>W</i> , target(<i>W</i>).response = ACCEPTED, and all other PENDING targets → CLOSED.
BR-03	Request validity. Only the shift's current worker can request; the shift must be SCHEDULED and start in the future; at most one open (REQUESTED/ACCEPTED) request per shift; the deadline must be after now and no later than the shift start.
BR-04	Approval effect. Approve: shift.workerId = acceptor; shift.originalWorkerId keeps the first worker (set only if null); request → APPROVED. Reject: request → REJECTED, shift unchanged. The owner can approve only an ACCEPTED request.

ID	Rule
BR-05	Week. A week runs Monday 00:00 to Sunday 24:00 (ISO week). Weekly records are keyed by the Monday date.
BR-06	Holiday-allowance eligibility. Eligible iff (a) weekly contractual hours ≥ 15 , and (b) perfect attendance: every shift of the week whose original worker is W and that was not given away was WORKED (not ABSENT). Shifts W gave away by an APPROVED substitution count according to Workplace.subAttendancePolicy: EXCUSED (default; the working day was changed by agreement, so it does not break attendance) or ABSENT (it breaks attendance). Contractual hours = sum of W's fixed-schedule hours per week. Substitute shifts W took do not change contractual hours. (The law averages 15 h over four weeks; the fixed schedule is constant, so the average equals the weekly value.)
BR-07	Holiday-allowance hours. hours = $\min(\text{contractualHours}, 40) / 40 \times 8$, rounded to 2 decimals; pay = hours $\times$ effective hourly wage, rounded down to the won.
BR-08	Premium pay. Applies only if Workplace.regularEmployees ≥ 5 . Then +50 % of wage for (i) daily hours beyond 8, (ii) weekly hours beyond 40 not already counted in (i), (iii) night hours 22:00–06:00. The night premium adds to the overtime premium. If regularEmployees < 5 , premium = 0.
BR-09	Minimum-wage check. Warn if worker.hourlyWage $< \text{MinimumWage}(\text{year}).\text{hourly} \times (0.9$ if BR-10 applies else 1). The payroll still computes with the worker's wage; the warning is shown on the payroll row and blocks confirmation until acknowledged.
BR-10	Probation reduction. The minimum may be reduced to 90 % only if all hold: contract length ≥ 1 year (contractEnd – contractStart ≥ 365 days, or contractEnd empty = open-ended); the date is within 3 months of contractStart and $\leq$ probationEnd; simpleLabor = false.
BR-11	Monthly payroll. Month = calendar month. Base pay = Σ actual hours of WORKED shifts in the month $\times$ wage (if no attendance is recorded yet, scheduled hours are used and the row is flagged "estimated"). Holiday allowance = Σ holiday pay of weeks whose Sunday falls in the month. Total = base + holiday + premium. Amounts are integers in KRW, rounded down.
BR-12	Expiry. A REQUESTED or ACCEPTED request whose deadline $\leq$ now, or whose shift start $\leq$ now, becomes EXPIRED.

4.3 Use-case descriptions

Each use case is described in the same structure: actors, preconditions, main flow, alternative and exception flows, postconditions, business rules and related functional requirements. Every use case except UC-11 includes UC-13 (the user must be signed in); this precondition is not repeated.

Table 6. UC-01 Register worker and fixed schedule

Actors	Owner
Preconditions	The owner is signed in.

Main flow	1. The owner opens <i>Staff</i> and chooses to add a worker. 2. The owner enters name, phone, hourly wage, contract start and end, optional probation end, the simple-labour flag and the "Currently employed" (active) flag. 3. The system validates the input and stores the worker. 4. Under "Fixed shifts" the owner picks weekdays and sets start and end times. 5. The system checks that the new fixed shift does not overlap another fixed shift of the same worker and stores it. 6. The system shows the worker's fixed schedule and weekly contractual hours ("Contract of 16 h a week").
Alternative / exception flows	3a. A required field is missing or the wage is not a positive integer: the system names the field and the fix; nothing is stored. 5a. The fixed shift overlaps an existing one: the system rejects it and shows the conflicting shift. 1a. The owner edits an existing worker or deactivates them; an inactive worker is no longer included in generated weeks or candidate lists.
Postconditions	Worker and FixedSchedule rows exist; the worker is included in the next generated week.
Business rules / FR	– (contractual hours feed BR-06) FR-01, FR-02

Table 7. UC-02 Register availability

Actors	Worker
Preconditions	The worker is signed in and active.
Main flow	1. The worker opens <i>Me</i>. 2. Under "When you can cover" the system shows the worker's availability slots by weekday. 3. The worker adds a weekday with start and end time, or removes one, and chooses <i>Save availability</i>. 4. The system validates and stores the change.
Alternative / exception flows	4a. End time is not after start time: the system rejects the slot and explains the rule.
Postconditions	Availability rows are updated; they are used for the next candidate selection.
Business rules / FR	BR-01 (input) FR-03

Table 8. UC-03 Generate weekly schedule

Actors	Owner
Preconditions	At least one active worker with a fixed schedule exists.
Main flow	1. The owner opens <i>Schedule</i> and navigates to a week (Monday to Sunday). 2. If the week has no shifts yet, the system generates it. 3. The system creates one SCHEDULED shift per fixed-schedule entry of every active worker, skipping entries that already have a shift in that week. 4. The system shows the week with seven day columns, shifts ordered by start time.
Alternative / exception flows	3a. The week is already generated: no duplicate is created; the week is shown unchanged. 4a. The owner chooses <i>Add shift</i> or edits a single shift; the system validates and stores the change. 1a. A worker opens <i>Schedule</i>: the week is read-only, the worker's own shifts are outlined in blue, and <i>Find a substitute</i> is offered instead of <i>Add shift</i>.
Postconditions	Shift rows for the week exist exactly once per fixed-schedule entry.
Business rules / FR	BR-05 FR-04, FR-05

Table 9. UC-04 Request substitute

Actors	Worker (requester); System (candidate selection)
Preconditions	The worker is the current worker of a SCHEDULED shift that starts in the future and has no open request.

Main flow	1. The worker chooses <i>Find a substitute</i> (on <i>Schedule</i>, on <i>Swaps</i>, or on the next-shift card of <i>Home</i>). 2. The system leads the worker through three steps: the shift (one of the worker's own future shifts), an optional reason, and the reply deadline, pre-filled with the earlier of now + 24 h and the shift start. 3. The system finds the eligible candidates (BR-01) «include» <i>Find eligible candidates</i>. 4. On the last step ("Here's what we'll send") the system shows the shift, the reason, the deadline and the candidates who will be notified; the worker chooses <i>Send swap request</i>. 5. The system checks request validity (BR-03). 6. In one transaction the system stores the request as REQUESTED, one PENDING target per candidate, and one REQUEST_RECEIVED notification per candidate. 7. The request appears on <i>Swaps</i> under "Your requests" with the progress step "Requested".
Alternative / exception flows	2a. A shift that already has an open request, or has started, is not offered (BR-03). 5a. BR-03 fails at sending time (for example the deadline has become invalid): the system states the cause and the fix; nothing is stored. 3a. No worker is eligible: the request is still stored when sent, the owner receives a NO_CANDIDATE notification, and the requester is told that no eligible worker exists. Extension points: deadline reached (UC-11); requester cancels (UC-12).
Postconditions	SubRequest is REQUESTED; each candidate has a PENDING target and an unread notification.
Business rules / FR	BR-01, BR-03 FR-06, FR-07, FR-08, FR-21

Table 10. UC-05 Respond to substitute request (accept / decline)

Actors	Worker (candidate)
Preconditions	The candidate has a PENDING target on a request.
Main flow	1. The candidate opens <i>Home</i>; the card under "Swap requests for you" shows the requester, the shift, the reason, the reply deadline and the effect on the candidate's own week, with <i>Accept</i> and <i>Decline</i>. The same request is listed on <i>Swaps</i> under "Requests for you". 2. The candidate chooses <i>Accept</i>. 3. In one transaction the system re-reads the request status. It is REQUESTED, so the system sets the request to ACCEPTED, records the acceptor, sets the candidate's target to ACCEPTED and all other PENDING targets to CLOSED. 4. The system notifies the requester and the owner (REQUEST_ACCEPTED) and the closed candidates (TARGET_CLOSED). 5. On the requester's <i>Swaps</i> screen the request moves to the progress step "Accepted".
Alternative / exception flows	2a. The candidate chooses <i>Decline</i>: the target becomes DECLINED; the request stays REQUESTED for the other candidates. 3a. The request is no longer REQUESTED (another candidate accepted first): the acceptance is refused with an explanatory message that names the worker who accepted first (FR-09). 3b. The request has expired or been cancelled: the acceptance is refused with that reason.
Postconditions	At most one acceptor per request; no PENDING target remains once the request is ACCEPTED.
Business rules / FR	BR-02 FR-09, FR-10, FR-21

Table 11. UC-06 Approve or reject substitution

Actors	Owner
Preconditions	A request is ACCEPTED.
Main flow	1. The owner opens <i>Home</i>; the section "Swaps waiting for approval" lists every ACCEPTED request as a card. 2. For requester and acceptor the system computes contractual hours, scheduled hours of the week, holiday-allowance eligibility and holiday hours, before and after the swap (FR-11). 3. The card shows one line per worker with the change in weekly hours and the effect on holiday allowance, and states any eligibility change. 4. The owner chooses <i>Approve</i>. 5. The system sets shift.workerId to the acceptor, keeps the original worker in shift.originalWorkerId, and sets the request to APPROVED with the decision time. 6. The system recomputes the weekly summaries of both workers «include» UC-08. 7. The system notifies both workers (REQUEST_APPROVED).

Alternative / exception flows	4a. The owner chooses <i>Reject</i> : the request becomes REJECTED, the shift is unchanged, both workers receive REQUEST_REJECTED. 4b. The request expired before the decision (BR-12): the decision is refused and the list is refreshed.
Postconditions	Approved: the shift belongs to the acceptor, and both weekly summaries are current. Rejected: no schedule change.
Business rules / FR	BR-04, BR-06, BR-07 FR-11, FR-12, FR-21

Table 12. UC-07 Record and confirm attendance

Actors	Worker, Owner
Preconditions	The shift has started (it is in the past relative to the demo clock).
Main flow	1. The worker opens <i>Me</i>; the “Attendance” section lists the worker’s shifts with their state (for example “Upcoming” or “Confirmed 18:00–23:00”). 2. For a shift that has ended, the worker enters actual clock-in and clock-out time. 3. The system stores the attendance with confirmed = 0. 4. The owner reviews the unconfirmed records. 5. The owner confirms a record; the system sets confirmed = 1 and the shift to WORKED. 6. The system recomputes the weekly summary of that worker (UC-08).
Alternative / exception flows	2a. Clock-out is not after clock-in (for a shift that does not cross midnight): the system rejects the entry. 5a. The worker did not work: the owner marks the shift ABSENT; perfect attendance for that week is broken.
Postconditions	The shift is WORKED or ABSENT; the weekly summary reflects it.
Business rules / FR	BR-06 (input) FR-13

Table 13. UC-08 Compute weekly hours and holiday allowance

Actors	Owner; System (when included by UC-06, UC-07 or UC-09)
Preconditions	The shifts of the week are generated.
Main flow	1. The owner opens <i>Pay</i>, tab <i>Weekly</i>, and picks a week. 2. For each active worker the system computes contractual hours (sum of fixed-schedule hours), scheduled hours (shifts the worker holds this week) and actual hours (confirmed attendance). 3. The system determines perfect attendance, applying the workplace policy (EXCUSED or ABSENT) to shifts given away by approved substitution. 4. The system determines eligibility (BR-06) and holiday hours (BR-07) and stores one WeeklySummary row per worker and week. 5. The system shows one card per worker with the eligibility, the holiday pay if eligible, and a short reason if not (for example “under 15 h”).
Alternative / exception flows	2a. The week is not generated: the system shows that the week has no shifts; the owner generates it in UC-03 first.
Postconditions	WeeklySummary rows for the week are current (unique per worker and week).
Business rules / FR	BR-05, BR-06, BR-07 FR-14

Table 14. UC-09 Generate and confirm monthly payroll

Actors	Owner
Preconditions	The owner is signed in; the minimum wage for the year of the month exists.

Main flow	1. The owner opens <i>Pay</i>, tab <i>Monthly</i>, and picks a month. 2. The system builds the draft; <i>Rebuild draft</i> builds it again after changes. 3. The system recomputes the weekly summaries of the weeks whose Sunday falls in the month «include» UC-08. 4. For each worker the system computes hours and base pay, holiday pay, premium pay and total (BR-08, BR-11), and checks the minimum wage (BR-09, BR-10). 5. The system stores the rows as DRAFT and shows them with status chips (Draft, Estimated, Probation minimum 90 %, Below minimum wage). 6. The owner chooses <i>Acknowledge</i> on each minimum-wage warning and then <i>Confirm <month> pay</i>. 7. The system sets the rows to CONFIRMED; they become read-only.
Alternative / exception flows	5a. Attendance is missing for some shifts: scheduled hours are used and the row is flagged "Estimated". 6a. A warning is not acknowledged: the confirm button stays disabled and the screen states how many warnings must be acknowledged. 2a. The month already has DRAFT rows: they are replaced; CONFIRMED rows are not changed.
Postconditions	One Payroll row per worker and month, DRAFT or CONFIRMED.
Business rules / FR	BR-08, BR-09, BR-10, BR-11 FR-15, FR-16

Table 15. UC-10 Manage workplace settings and minimum wage table

Actors	Owner
Preconditions	The owner is signed in.
Main flow	1. The owner opens <i>Settings</i>. 2. The system shows the store name, the number of regular employees, the policy for handed-over shifts ("Do not count as absence" = EXCUSED, the default, or "Count as absence" = ABSENT) and the minimum wage by year. 3. The owner edits values, adds or removes minimum-wage rows, and chooses <i>Save</i>. 4. The system validates and stores the settings.
Alternative / exception flows	4a. Regular employees is not a non-negative integer, or a minimum wage row has a duplicate year: the system rejects the change and explains it.
Postconditions	Later weekly summaries use the new policy; later payroll drafts use the new employee count and minimum wages.
Business rules / FR	BR-06 (policy), BR-08 (threshold), BR-09 (minimum wage) FR-17

Table 16. UC-11 Expire overdue substitute request

Actors	System Clock
Preconditions	A request is REQUESTED or ACCEPTED.
Main flow	1. The System Clock runs the expiry check (on application start, on every screen change and after the demo clock is changed). 2. The system finds open requests whose deadline $\leq$ now or whose shift start $\leq$ now. 3. For each, the system sets the request to EXPIRED and all PENDING targets to CLOSED. 4. The system notifies the requester and the owner (REQUEST_EXPIRED).
Alternative / exception flows	2a. No request is overdue: nothing changes.
Postconditions	No open request has a past deadline; the shift stays with the requester.
Business rules / FR	BR-12 FR-18, FR-21

Table 17. UC-12 Cancel own substitute request

Actors	Worker (requester)
Preconditions	The worker's request is REQUESTED or ACCEPTED.

Main flow	1. The worker opens <i>Swaps</i>; under “Your requests” each request shows its progress (Requested → Accepted → Approved). 2. The worker chooses <i>Cancel request</i> on an open request. 3. The system sets the request to CANCELLED and closes all PENDING targets. 4. The system notifies the candidates and the acceptor, if any (REQUEST_CANCELLED).
Alternative / exception flows	2a. The request is already APPROVED, REJECTED, EXPIRED or CANCELLED: no <i>Cancel request</i> button is shown.
Postconditions	The request is CANCELLED; the shift stays with the requester.
Business rules / FR	– FR-19, FR-21

Table 18. UC-13 Sign in by role (demo account selection)

Actors	Worker, Owner
Preconditions	The application is open in a browser.
Main flow	1. The system shows the demo page: a three-step tour (see Lee Seoyeon's request, accept as Choi Minho, approve as owner Park Jiyoung) and the list of all accounts (owner and four workers) with role labels. There are no passwords. 2. The user starts the tour or chooses an account. 3. The system keeps the choice for the browser session and shows only the menus of that role: <i>Home, Schedule, Swaps, Me</i> for a worker; <i>Home, Schedule, Staff, Pay</i> for the owner, who also has <i>Settings</i>.
Alternative / exception flows	3a. The user signs out: the session choice is cleared and the sign-in screen is shown.
Postconditions	A signed-in user with role OWNER or WORKER.
Business rules / FR	– (NFR-11 role guard) FR-20

5. Non-functional requirements specification

Non-functional requirements are classified as in Chapter 8 into *product* requirements (how the delivered system must behave), *organisational* requirements (which follow from the team's and the course's policies) and *external* requirements (which come from outside the system and the project, here mainly law). Each requirement is stated so that it can be verified, and the verification method is given.

Table 19. Non-functional requirements

ID	Sub-class	Requirement	Verification
Product requirements			
NFR-01	Usability	A candidate can accept a request in at most 2 clicks from the inbox.	UI walkthrough
NFR-02	Usability	All screens are usable at 360 px width (mobile) and 1440 px (desktop) without horizontal scrolling.	Browser check at both widths
NFR-03	Efficiency / performance	With 20 workers and 12 weeks of shifts, any screen renders in under 1 second on a mid-range laptop.	Timed load with seeded data
NFR-04	Reliability	“First acceptance wins” is enforced inside one database transaction that re-checks the request state; two acceptances can never both succeed.	Unit test TC-05x

ID	Sub-class	Requirement	Verification
NFR-05	Reliability	Data persists across page reloads (browser storage) and can be reset to seed data.	Manual test
NFR-06	Portability	Runs in current Chrome, Edge, Safari and Firefox without installation or server.	Manual test
Organisational requirements			
NFR-07	Delivery	Working system deployed by Week 14 (2026-12-04).	Deployed URL
NFR-08	Implementation	Static web application (HTML/CSS/JavaScript ES modules) with an embedded SQLite database (sql.js, WebAssembly); built by vibe coding from the final prompt; hosted on GitHub Pages.	Repository
NFR-09	Standards	Models follow UML 2.5 notation; code identifiers are in English.	Review
External requirements			
NFR-10	Legislative	Pay rules follow the Minimum Wage Act (2026 minimum hourly wage KRW 10,320) and the Labor Standards Act (Art. 18 ☺, Art. 55, Enforcement Decree Art. 30; Art. 11 / 56 premium threshold).	Unit tests TC-08x, TC-09x
NFR-11	Privacy	A worker's phone number and wage are visible only to the owner and that worker (Personal Information Protection Act principle of minimum disclosure).	Role test
NFR-12	Ethical	The system does not collect location data; attendance is self-reported and owner-confirmed.	Design review

Two of these requirements shape the design directly. NFR-04 means that candidate responses cannot be handled by simply reading and then writing the request; the status check and the update must be one transaction. NFR-08 means that there is no server to enforce access rules, so NFR-11 is enforced by the role guard of the user interface, and the application is presented as a single-store demonstration system.

6. Scenarios

Chapter 8 defines a scenario as a real-life example of how the system is used, with a starting situation, a normal flow, what can go wrong, other concurrent activities and the state at the end. The scenarios below use the demo store of Section 2.3. Unless stated otherwise, the demo clock starts at Monday 2026-09-28 09:00 and the policy is EXCUSED. Each alternative sequence is written as a separate scenario, as Chapter 8 recommends.

S1. Lee Seoyeon requests a substitute for her midterm exam (UC-04)

Starting situation. Lee Seoyeon works Monday and Wednesday 18:00–23:00. She has a midterm exam on Wednesday 2026-09-30 and cannot work that evening.

Normal flow.

1. Seoyeon signs in and opens *Schedule* for the week of 2026-09-28. Her own shifts are outlined in blue.
2. She chooses *Find a substitute* and, in the first step, picks her Wednesday 30 September 18:00–23:00 shift.
3. She enters the reason “Midterm exam” and the deadline 2026-09-29T21:00, and submits.
4. The system checks BR-03: the shift is hers, SCHEDULED, in the future, has no open request, and the deadline is before the shift start.
5. The system applies BR-01. Choi Minhoo has no shift on Wednesday and is available Wednesday 17:00–23:00, which covers 18:00–23:00. Kang Doyun has no shift on Wednesday and is available Wednesday 18:00–23:00. Jung Hana is not available on Wednesday. Both Minhoo and Doyun are eligible.
6. The system stores the request as REQUESTED, two PENDING targets, and a REQUEST_RECEIVED notification for each of the two candidates.
7. On *Swaps* the request appears under “Your requests” at the step “Requested”. On *Schedule* her Wednesday shift now has a dashed amber outline and the chip “Looking for cover” (Figures 9 and 10).

What can go wrong. If Seoyeon enters a deadline after 2026-09-30 18:00, the system refuses and says that the deadline must be no later than the shift start.

End state. One open request with two candidates. This is the state of the demo data.

S2. Choi Minhoo accepts first; Kang Doyun is refused (UC-05)

Starting situation. The end state of S1. Both candidates have an unread notification.

Normal flow.

1. Minhoo signs in. The bell in the header shows one unread notification, and *Home* lists the request under “Swap requests for you” (Figure 12).
2. The card shows Seoyeon's Wednesday shift, the reason, the reply deadline and the effect on his own week: 16 h → 21 h, holiday allowance unchanged (KRW 33,024). He chooses *Accept*, one tap on *Home*.
3. In one transaction the system reads the request status (REQUESTED), sets it to ACCEPTED with Minhoo as acceptor, sets Minhoo's target to ACCEPTED and Doyun's target to CLOSED.
4. Seoyeon and the owner receive REQUEST_ACCEPTED; Doyun receives TARGET_CLOSED.
5. Doyun can no longer accept the request. An acceptance attempt is refused with a message that Choi Minhoo accepted first (FR-09).
6. On Seoyeon's *Swaps* screen the request moves to the step “Accepted” with the text “Minhoo accepted. Waiting for the owner's approval.” (Figure 13).

Concurrent activity. If Doyun had pressed *Accept* at almost the same moment as Minhoo, his transaction would read the status after Minhoo's transaction committed. The status would be ACCEPTED, not REQUESTED, so his acceptance is refused with the same message (BR-02, NFR-04, TC-05x).

End state. Request ACCEPTED, acceptor Choi Minhoo. The shift still belongs to Seoyeon until the owner decides.

S3. The owner approves the swap and sees Minhoo's hours change from 16 to 21 (UC-06, UC-08)

Starting situation. The end state of S2.

Normal flow.

1. Park Jiyoung signs in as owner. *Home* reads “1 thing to handle today” and lists the swap under “Swaps waiting for approval” (Figure 14).
2. The system compares both workers before and after the swap:

Table 20. Comparison for S3 (week of 2026-09-28, policy EXCUSED)

Worker	Contract h (before → after)	Scheduled h this week	Holiday allowance	Holiday h
Lee Seoyeon (requester)	10 → 10	10 → 5	Not eligible → not eligible (contract hours below 15)	0 → 0
Choi Minho (acceptor)	16 → 16	16 → 21	Eligible → eligible	3.2 → 3.2

The card summarises this in two lines: “Seoyeon: this week 10 h → 5 h · no holiday allowance (contract 10 h a week)” and “Minho: this week 16 h → 21 h · holiday allowance unchanged (KRW 33,024)”. No eligibility changes.

3. She chooses *Approve*. The shift's worker becomes Minho; its original worker is recorded as Seoyeon; the request becomes APPROVED.
4. The system recomputes both weekly summaries. Minho's holiday allowance for the week remains $16 / 40 \times 8 = 3.2$ h, that is $3.2 \times 10,320 = \text{KRW } 33,024$, provided he works all his shifts of the week.
5. Seoyeon and Minho receive REQUEST_APPROVED.
6. On *Schedule*, the Wednesday shift has an amber left edge and stripe pattern, shows “Choi Minho” in bold and “Lee Seoyeon” struck through beneath it (Figure 15).

What can go wrong. If the owner chooses *Reject*, the request becomes REJECTED, the shift stays with Seoyeon and both receive REQUEST_REJECTED. Seoyeon then has to request again or work the shift.

End state. Minho holds the Wednesday shift; the weekly summaries are current.

S4. A request expires at its deadline (UC-11)

Starting situation. The end state of S1: the request is REQUESTED with deadline 2026-09-29T21:00. In this variant neither candidate answers.

Normal flow.

1. The demo clock is advanced from Monday 09:00 with *+1 day* to Tuesday 2026-09-29 09:00. The request is still REQUESTED.
2. The clock is advanced with *+1 hour* until it reaches 2026-09-29T21:00.
3. On the next screen change the System Clock runs the expiry check. The deadline $\leq$ now, so the request becomes EXPIRED and both PENDING targets become CLOSED (BR-12).
4. Seoyeon and the owner receive REQUEST_EXPIRED. Minho and Doyun can no longer accept the request.

What can go wrong. The same happens if the request is ACCEPTED but the owner does not decide before the deadline. The owner then sees the request disappear from the ACCEPTED list.

End state. The Wednesday shift stays with Seoyeon. She may create a new request, because no open request exists for the shift any more, as long as the new deadline is before 2026-09-30 18:00.

S5. Jung Hana asks for a Sunday substitute and nobody is eligible (UC-04, FR-08)

Starting situation. Demo clock Monday 2026-09-28 09:00. Jung Hana cannot work Sunday 2026-10-04 10:00–16:00.

Normal flow.

1. Hana requests a substitute for that shift.
2. The system applies BR-01. No worker has an availability slot on Sunday, so the candidate list is empty.
3. The request is stored as REQUESTED. The owner receives a NO_CANDIDATE notification. Hana sees the message that no eligible worker exists.

End state. The owner now knows, without any telephone call, that the shift has to be solved by other means. If nothing changes, the request expires at its deadline as in S4.

S6. Month-end payroll for September 2026 (UC-09)

Starting situation. The owner has finished the month and opens *Pay*, tab *Monthly*, for September 2026. The store has 4 regular employees.

Normal flow.

1. The system builds the draft with one row per worker (Figure 20).
2. Premium pay is 0 for every worker, because the store has fewer than 5 regular employees (BR-08).
3. Holiday pay counts the weeks whose Sunday falls in September. For the week of 2026-09-21 (Sunday 2026-09-27), all shifts have confirmed attendance. Minho is eligible (16 h contract) and receives KRW 33,024 for that week. Seoyeon (10 h), Hana (12 h) and Doyun (12 h) are below 15 h and receive no holiday pay for it.
4. Kang Doyun's wage is KRW 10,000. BR-10 does not apply to him: his contract (2026-08-15 to 2026-11-15) is shorter than one year and his work is simple labor. His applicable minimum is therefore the full KRW 10,320. His row shows a minimum-wage warning (BR-09).
5. For Jung Hana the system checks BR-10: her contract is open-ended, 2026-09 is within three months of her contract start 2026-09-01 and before her probation end 2026-11-30, and her work is not simple labor. The reduced minimum $10,320 \times 0.9 = \text{KRW } 9,288$ applies. Her wage of KRW 10,320 is above it, so there is no warning.
6. Rows that include shifts without recorded attendance use scheduled hours and are flagged “estimated” (BR-11).
7. The draft total for all workers is KRW 703,584. *Confirm September pay* stays disabled with the note “Acknowledge every minimum-wage warning (1) before confirming.” The owner chooses *Acknowledge* on Doyun's row and then confirms. All rows become CONFIRMED and read-only.

What can go wrong. The warning does not change the amount; the system still pays Doyun's contract wage. The owner has to correct the wage in UC-01 and regenerate the draft before confirming if the error is to be fixed in this payroll.

End state. Confirmed payroll for September 2026.

S7. Policy variant ABSENT: a handed-over shift breaks perfect attendance (UC-10, UC-06)

Starting situation. The owner changes the policy for handed-over shifts in *Settings* from “Do not count as absence” (EXCUSED) to “Count as absence” (ABSENT) (Figure 22). Choi Minhø asks for a substitute for his Saturday 2026-10-03 12:00–18:00 shift.

Normal flow.

1. BR-01 selects Lee Seoyeon (available Saturday 10:00–22:00, no Saturday shift) and Jung Hana (available Saturday 10:00–18:00, no Saturday shift). Kang Doyun is not eligible because his own Saturday 10:00–16:00 shift overlaps.
2. Hana accepts.
3. The owner opens the approval card on *Home*. Minhø's contractual hours stay 16 and his scheduled hours fall from 16 to 10. Under the ABSENT policy the given-away shift breaks his perfect attendance, so his eligibility changes from *eligible* to *not eligible*. The card states this change. Hana's scheduled hours rise from 12 to 18; her contract is 12 h, so she is not eligible before or after.
4. The owner can now decide with this information. Under the default EXCUSED policy the same swap would leave Minhø eligible, with 3.2 holiday hours.

End state. Depends on the owner's decision. The scenario shows why BR-06 is a setting and not a fixed rule (risk R1 of the proposal).

7. UML diagrams for requirements analysis

This section presents the analysis models. They are drawn in UML 2.5 notation [8]. Following Chapter 2, they show the system from several perspectives: the external perspective (use case diagram), the behavioural perspective (activity and state machine diagrams) and the structural perspective (analysis class diagram). Sequence diagrams belong to the design and are given in the final report.

7.1 Use case diagram

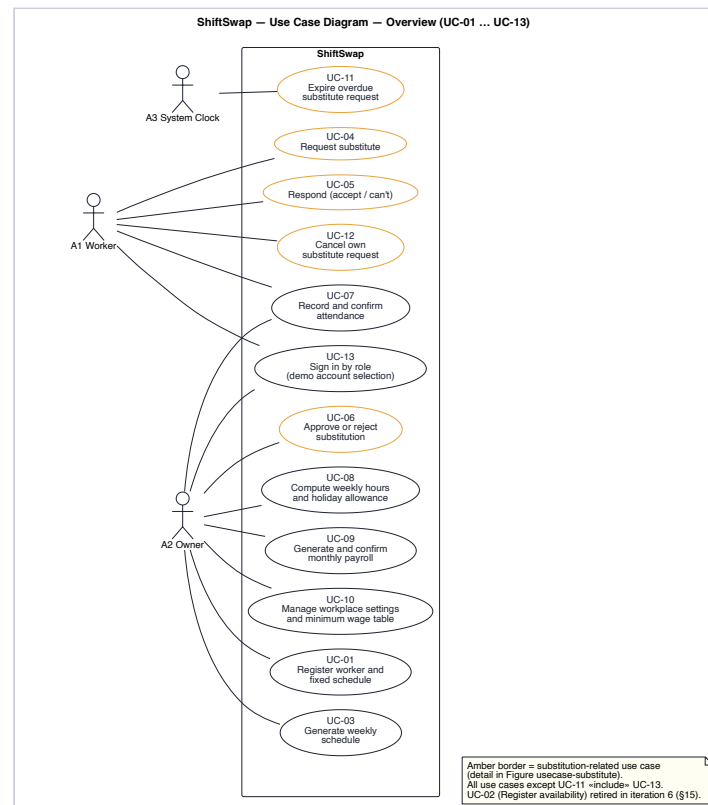


Figure 1. Use case diagram of ShiftSwap

Figure 1 shows the three actors and the thirteen use cases. The Worker and the Owner are primary actors. The System Clock is a secondary actor that starts only UC-11. UC-04 includes UC-04a *Find eligible candidates*, which is the place of rule BR-01. UC-06 includes UC-08, because every approval recomputes the weekly summaries of two workers. UC-09 also includes UC-08, because payroll uses the weekly holiday allowance.

UC-11 and UC-12 extend UC-04. UC-11 applies at the extension point “deadline reached with no approved acceptance”. UC-12 applies when the requester withdraws the request. All use cases except UC-11 include UC-13, since a user must be signed in. The diagram contains the nine main functions of the proposal (UC-01 to UC-09) unchanged.

7.2 Activity diagrams

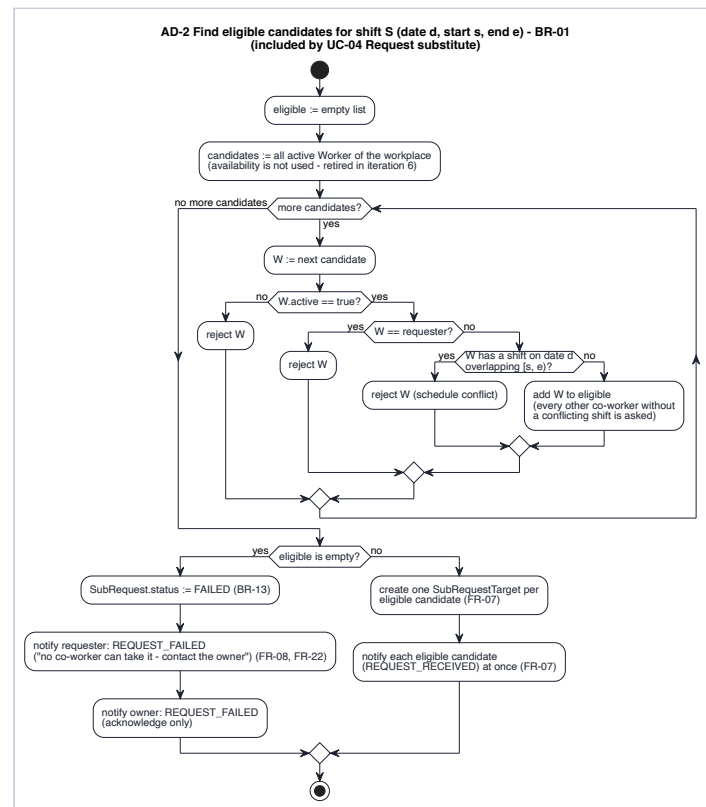


Figure 2. Activity diagram AD-2: candidate selection (BR-01)

Figure 2 shows how candidates are selected when a request is created. For each worker the system tests the four conditions of BR-01 in order: active, not the requester, no overlapping shift on the date, and an availability slot that covers the whole shift. A worker who fails a test is skipped. After the loop the diagram branches on the number of candidates: with one or more candidates, targets and REQUEST_RECEIVED notifications are created; with none, the owner receives NO_CANDIDATE (FR-08).

Scenario S1 follows the first branch with two candidates. Scenario S5 follows the second branch. Scenario S7 shows the overlap test rejecting a worker (Kang Doyun).

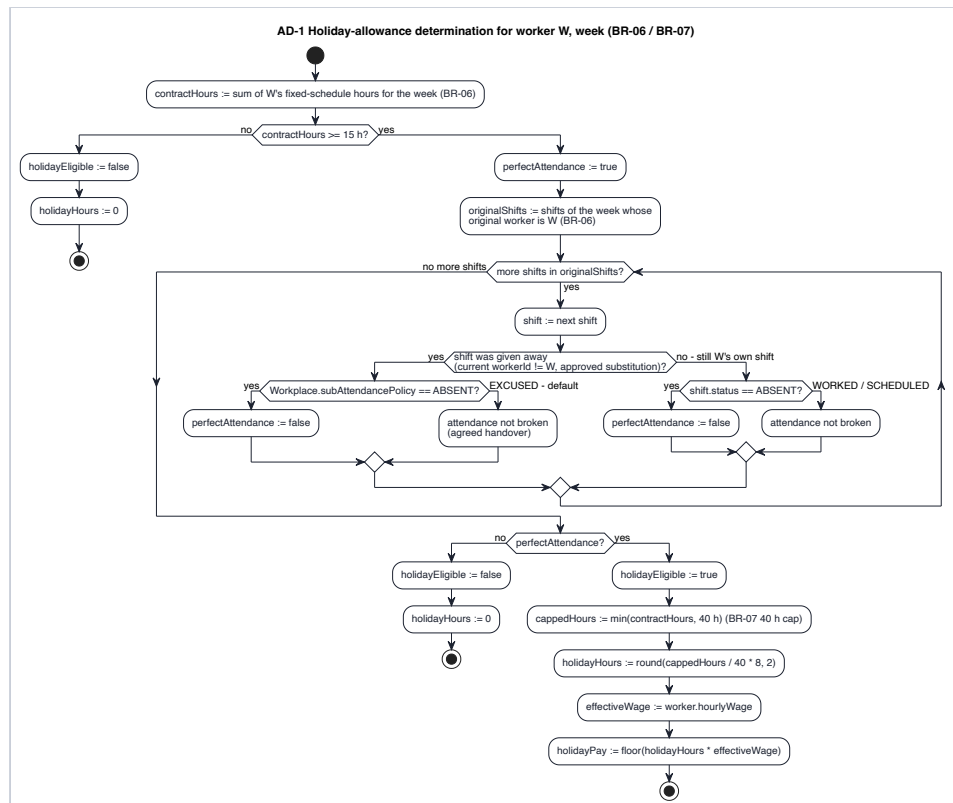


Figure 3. Activity diagram AD-1: holiday-allowance determination (BR-06, BR-07)

Figure 3 shows the weekly holiday-allowance decision for one worker. The first decision is the 15-hour test on contractual hours. The second is perfect attendance: no shift of the week that the worker originally held and still holds may be ABSENT. For a shift the worker gave away by an approved substitution, the diagram branches on the workplace policy. Under EXCUSED the shift is skipped; under ABSENT it breaks attendance. If the worker is eligible, holiday hours are $\min(\text{contractual hours}, 40) / 40 \times 8$.

The diagram makes the policy variant of scenario S7 visible. It also makes clear that substitute shifts a worker *took* do not enter the 15-hour test, because that test uses contractual hours only.

7.3 State machine diagrams

Chapter 8 requires that states in a requirements model are domain-significant, that all sequences from the scenarios are allowed, and that prohibited sequences are not allowed. The three diagrams below are read against scenarios S1 to S7 with these rules.

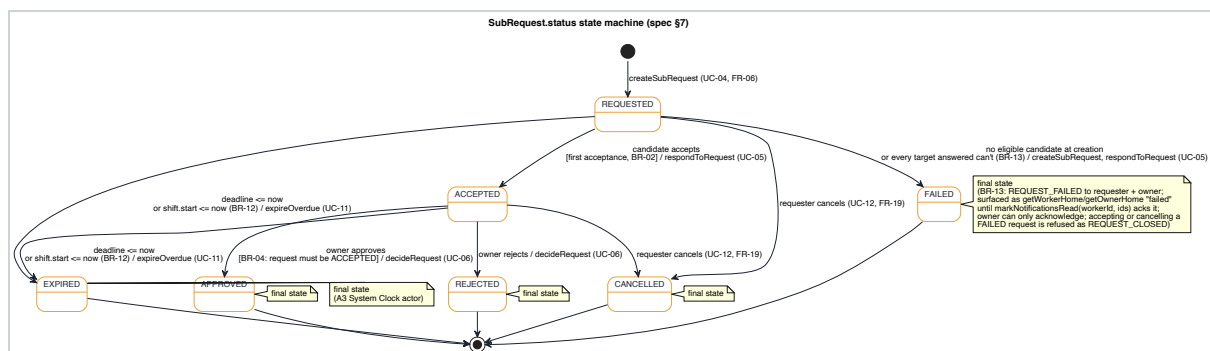


Figure 4. State machine diagram of SubRequest

Figure 4 shows the life of a substitute request. A request starts as REQUESTED. The first acceptance moves it to ACCEPTED (UC-05). The owner's decision moves it to APPROVED or REJECTED (UC-06). From REQUESTED or ACCEPTED it can also become EXPIRED (UC-11) or CANCELLED (UC-12). The four end states are final.

The diagram also shows what is prohibited. There is no transition from REQUESTED directly to APPROVED, so the owner cannot approve a request nobody accepted (BR-04). There is no transition out of ACCEPTED on a second acceptance, which is the state-machine form of BR-02. Scenarios S1–S4 each follow one path of this diagram.

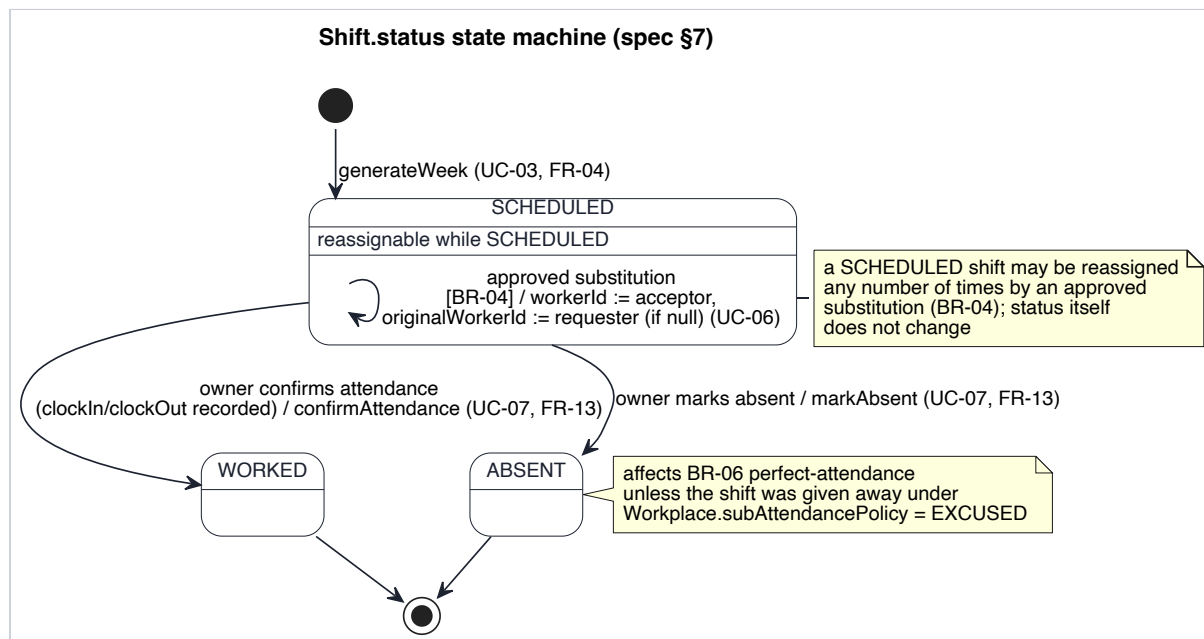


Figure 5. State machine diagram of Shift

Figure 5 shows that a shift is SCHEDULED when generated and becomes WORKED when the owner confirms attendance or ABSENT when the owner marks it so (UC-07). Reassignment on approval (BR-04) is a self-transition on SCHEDULED: the worker changes, the state does not. A shift that is WORKED or ABSENT can no longer be requested, because BR-03 requires a SCHEDULED shift in the future.

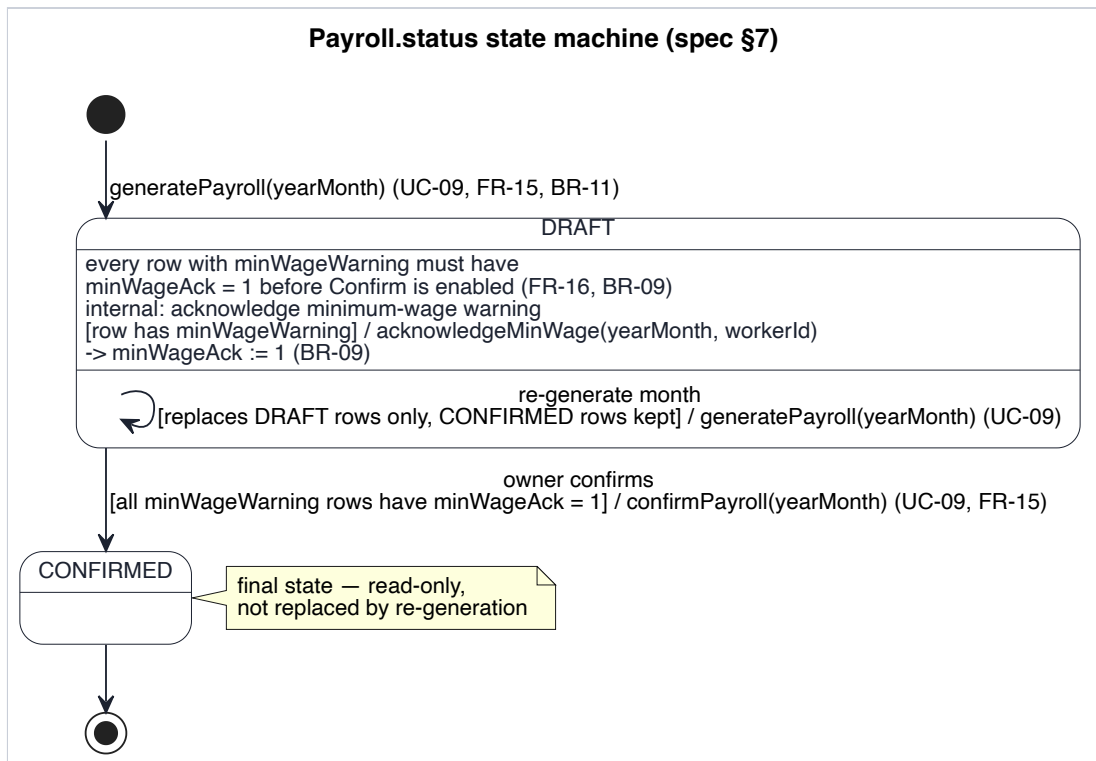


Figure 6. State machine diagram of Payroll

Figure 6 shows the two payroll states. A generated row is DRAFT. Re-generating the month replaces DRAFT rows. Confirmation is guarded: it is possible only when every minimum-wage warning of the month has been acknowledged (BR-09). A CONFIRMED row is final and read-only (FR-15), which scenario S6 ends with.

7.4 Analysis class diagram

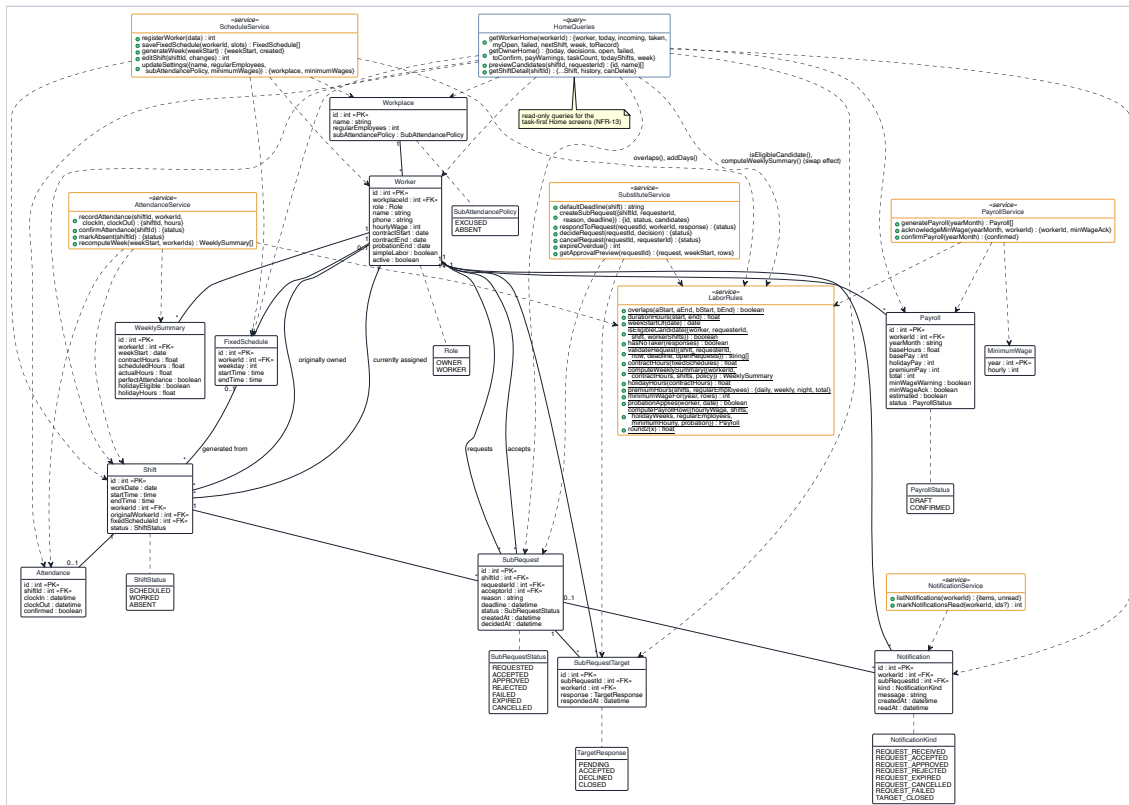


Figure 7. Analysis class diagram (domain view)

Figure 7 shows the twelve domain classes, their attributes and their associations. Workplace owns Workers and the settings used by the rules (regular employees, substitution-attendance policy). A Worker has FixedSchedule entries, which define the contractual hours, and Availability slots, which feed BR-01. Shifts are generated from FixedSchedule entries. A Shift refers to its current worker and, after a swap, to its original worker. A SubRequest belongs to one Shift, has one requester, at most one acceptor, and one SubRequestTarget per candidate. Attendance is at most one per Shift. WeeklySummary and Payroll are derived records per worker and week or month. MinimumWage holds one row per year, and Notification belongs to one recipient and optionally one request.

The lower part of the diagram shows the five service classes (ScheduleService, SubstituteService, AttendanceService, PayrollService, NotificationService) and the LaborRules rule object. At the analysis stage each service operation stands for one step of a use case; for example, SubstituteService.respondToRequest realises UC-05 and SubstituteService.expireOverdue realises UC-11. LaborRules groups the business rules BR-01 to BR-12 as operations that read no stored data, so they can be tested alone (Section 9.3). Each domain class corresponds to one of the 12 database tables, so the data requirement of the course (at least four tables) is met three times over. The design class diagram of the final report will refine these classes with the sequence diagrams.

8. Main user interface screens

The screens below were captured from the prototype on the demo data of Section 2.3, with the demo clock at Monday 2026-09-28 09:00. A worker has four screens: *Home*, *Schedule*, *Swaps* and *Me*. The owner has *Home*, *Schedule*, *Staff* and *Pay*, plus *Settings*. A *Demo tools* screen controls the demo clock and the demo data. On a desktop the screens are listed in a left navigation; on a phone they be-

come a bottom tab bar. Colour is used sparingly: blue marks the signed-in worker's own shifts and primary buttons, and amber is reserved for substitution (a dashed outline with “Looking for cover” for an open request, an amber edge and stripe for a handed-over shift). Status chips carry text, not only colour. The header of every screen shows the demo clock, the notification bell with its unread count, and the signed-in user.

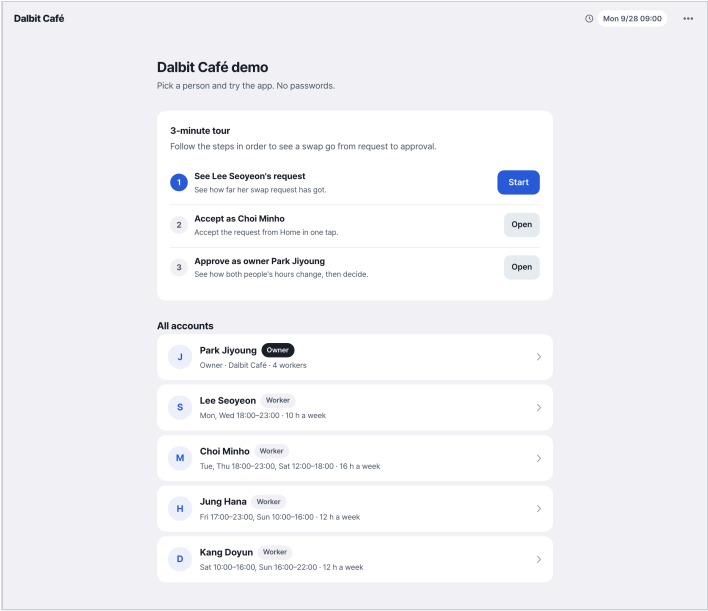


Figure 8. Sign-in (UC-13). The “Dalbit Café demo” page (“Pick a person and try the app. No passwords.”) offers a “3-minute tour” in three steps: 1 See Lee Seoyeon's request, 2 Accept as Choi Minho, 3 Approve as owner Park Jiyoung. Below, “All accounts” lists the owner (Dalbit Café, 4 workers) and the four workers with their fixed shifts and weekly hours (10, 16, 12 and 12 h).

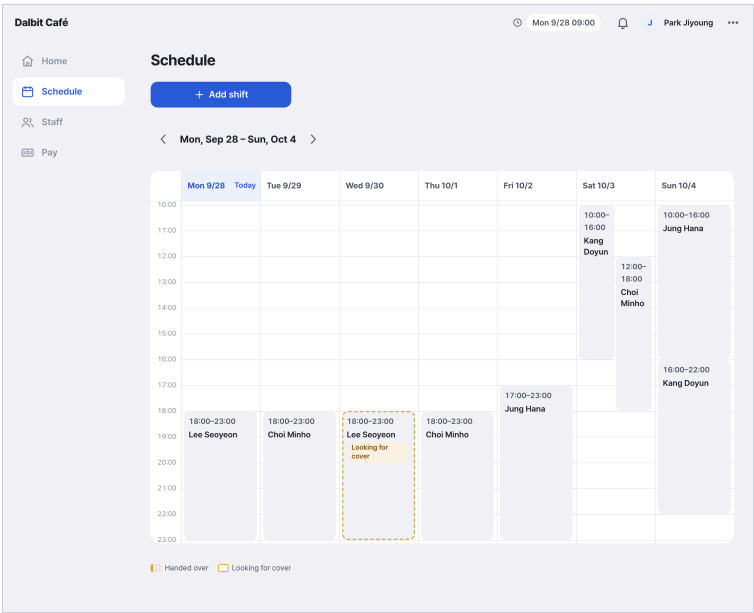


Figure 9. Schedule, owner view (UC-03). The week Mon, Sep 28 to Sun, Oct 4 in seven day columns from 10:00 to 23:00, with *Add shift* above. Lee Seoyeon's Wednesday 18:00–23:00 shift has a dashed amber outline and the chip “Looking for cover”. The legend explains “Handed over” and “Looking for cover”.

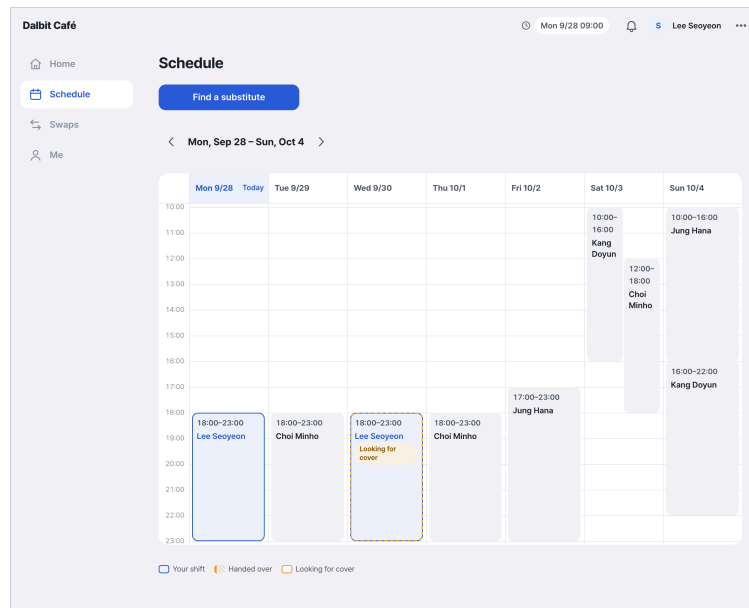


Figure 10. Schedule, worker view (UC-03, UC-04), signed in as Lee Seoyeon. Her own shifts (Mon and Wed 18:00–23:00) are outlined and shaded blue with her name in blue; the Wednesday shift is also “Looking for cover”. The worker sees *Find a substitute* instead of *Add shift*, and the navigation Home, Schedule, Swaps and Me.

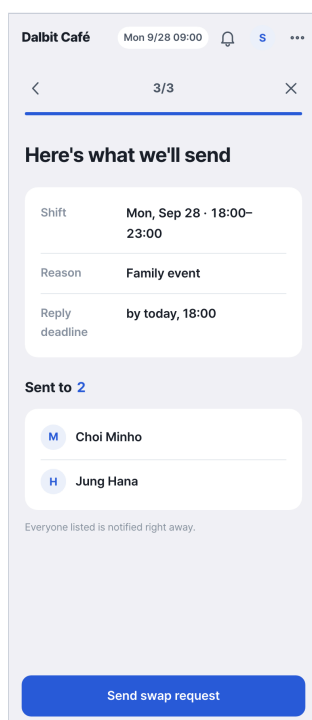


Figure 11. Request substitute, step 3 of 3 (UC-04), Lee Seoyeon. “Here’s what we’ll send”: shift Mon, Sep 28 18:00–23:00, reason “Family event”, reply deadline “by today, 18:00” (the shift start, earlier than now + 24 h). “Sent to 2”: Choi Minho and Jung Hana. Button: *Send swap request*.

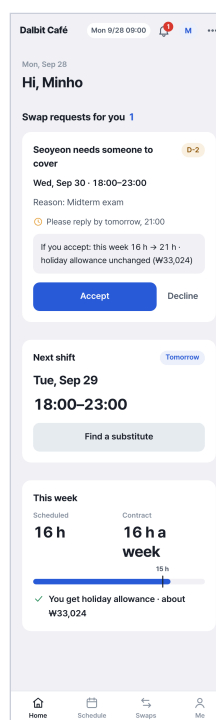


Figure 12. Home of Choi Minho (UC-05). Under “Swap requests for you 1”: “Seoyeon needs someone to cover”, Wed, Sep 30 18:00–23:00, reason “Midterm exam”, reply by tomorrow 21:00, and “If you accept: this week 16 h → 21 h · holiday allowance unchanged (₩33,024)”, with *Accept* and *Decline*. Below: next shift and this week’s hours.

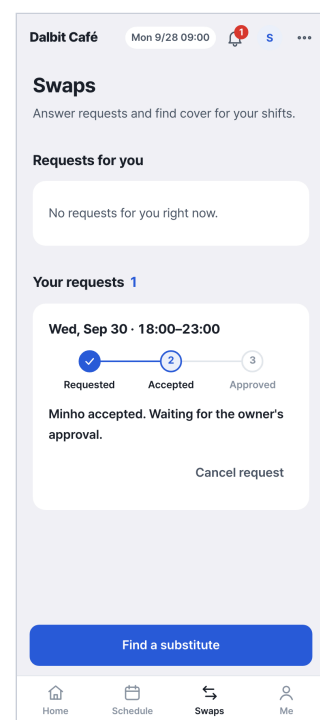


Figure 13. Swaps of Lee Seoyeon (UC-05, UC-12) after Minho accepted. “Your requests 1”: Wed, Sep 30 18:00–23:00 with the progress Requested → Accepted → Approved at step 2 and “Minho accepted. Waiting for the owner’s approval.” *Cancel request* and *Find a substitute* are available.

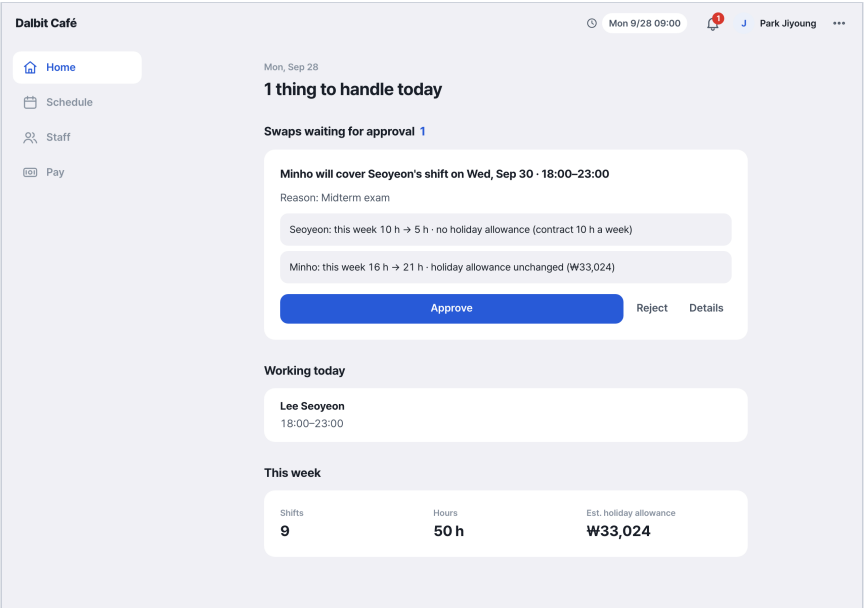


Figure 14. Home of the owner (UC-06, FR-11). “1 thing to handle today”. Under “Swaps waiting for approval 1”: “Minho will cover Seoyeon's shift on Wed, Sep 30 · 18:00–23:00”, reason “Midterm exam”, then one line per worker: Seoyeon this week 10 h → 5 h, no holiday allowance (contract 10 h a week); Minho 16 h → 21 h, holiday allowance unchanged (₩33,024). Buttons *Approve*, *Reject*, *Details*. Below: working today and this week (9 shifts, 50 h, estimated holiday allowance ₩33,024).

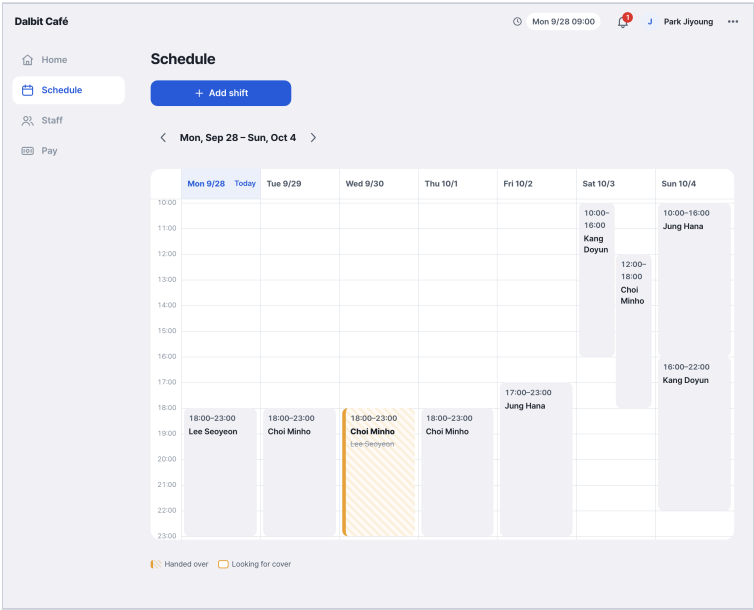


Figure 15. Schedule after approval (UC-06, BR-04). The Wednesday 09-30 18:00–23:00 shift shows “Choi Minho” in bold with “Lee Seoyeon” struck through beneath it, on an amber left edge with a diagonal amber stripe. The bell shows 1 unread notification.

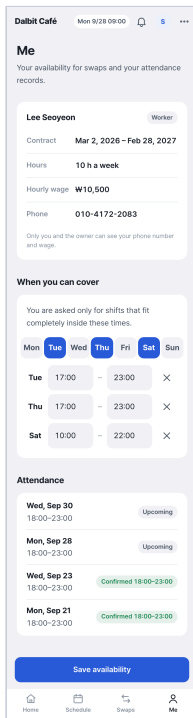


Figure 16. Me of Lee Seoyeon (UC-02, UC-07, NFR-11). Contract Mar 2, 2026 – Feb 28, 2027, 10 h a week, hourly wage ₩10,500 and phone, with the note that only she and the owner can see phone and wage. “When you can cover”: Tue 17:00–23:00, Thu 17:00–23:00, Sat 10:00–22:00. “Attendance”: Sep 30 and Sep 28 upcoming, Sep 23 and Sep 21 confirmed 18:00–23:00.

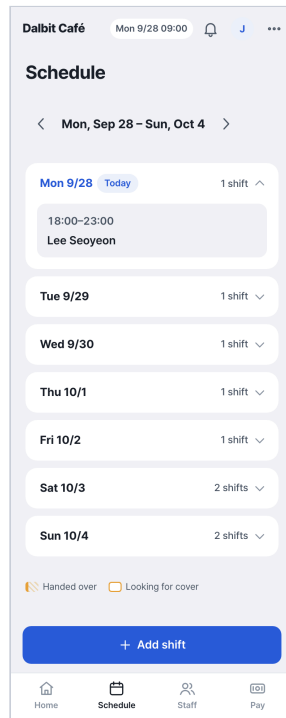


Figure 17. Schedule on a 390 px wide phone (NFR-02), owner view. The week becomes a list of days with the number of shifts per day; Mon 9/28 is expanded (Lee Seoyeon 18:00–23:00). *Add shift* sits above the bottom tab bar Home, Schedule, Staff, Pay.



Figure 18. The same step as Figure 11 with the language set to Korean: title, field labels, the reason, the deadline and the send button are in Korean; the date is shown in Korean format.

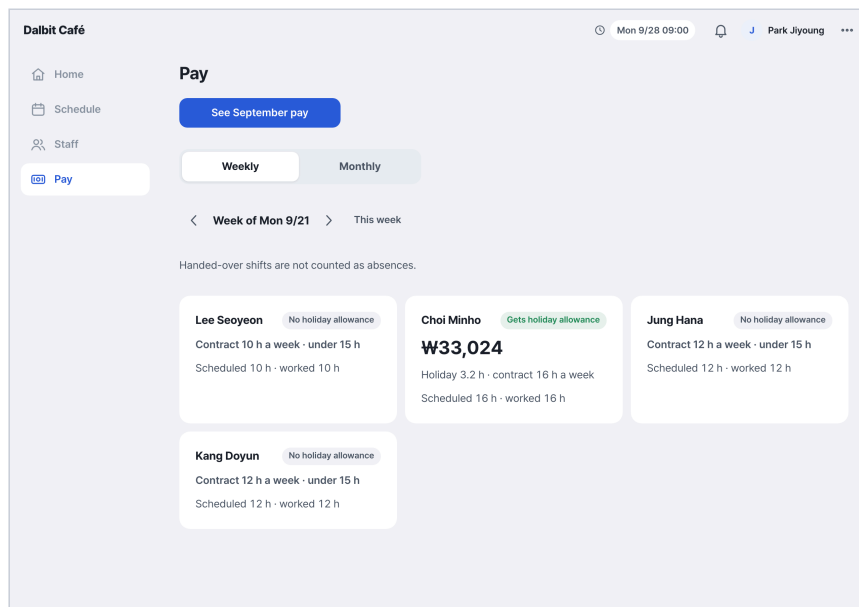


Figure 19. Pay, tab Weekly (UC-08, FR-14), week of Mon 9/21, with the note “Handed-over shifts are not counted as absences.” (EXCUSED). One card per worker: Choi Minho “Gets holiday allowance”, ₩33,024, holiday 3.2 h, contract 16 h a week, scheduled 16 h, worked 16 h. Lee Seoyeon (10 h), Jung Hana (12 h) and Kang Doyun (12 h): “No holiday allowance”, contract under 15 h, scheduled and worked hours equal.

Worker	Hours	Base pay	Holiday pay	Premium	Total	Status
Lee Seoyeon	20 h	₩210,000	₩0	₩0	₩210,000	Draft Estimated
Choi Minho	21 h	₩216,720	₩33,024	₩0	₩249,744	Draft Estimated
Jung Hana	12 h	₩123,840	₩0	₩0	₩123,840	Draft Probation minimum 90 %
Kang Doyun	12 h	₩120,000	₩0	₩0	₩120,000	Draft Below minimum wage
Doyun's hourly wage of ₩10,000 is below the 2026 minimum wage of ₩10,320					Acknowledge	
All workers					₩703,584	

Figure 20. Pay, tab Monthly (UC-09, FR-15, FR-16), September 2026. Lee Seoyeon 20 h, ₩210,000 (Draft, Estimated); Choi Minho 21 h, ₩216,720 + ₩33,024 holiday pay = ₩249,744 (Draft, Estimated); Jung Hana 12 h, ₩123,840 (Draft, Probation minimum 90 %); Kang Doyun 12 h, ₩120,000 (Draft, Below minimum wage) with "Doyun's hourly wage of ₩10,000 is below the 2026 minimum wage of ₩10,320" and *Acknowledge*. All workers ₩703,584. Premium is ₩0 (4 regular employees). *Confirm September pay* is disabled until the 1 warning is acknowledged.

Figure 21. Staff detail (UC-01), Choi Minho: name, phone, hourly wage 10320, contract 06/01/2026 – 12/31/2026, empty probation end, simple labour off, currently employed on, and fixed shifts Tue and Thu 18:00–23:00 and Sat 12:00–18:00: "Contract of 16 h a week."

Figure 22. Settings (UC-10): store name Dalbit Café, 4 regular employees ("With 5 or more, overtime and night premiums apply"), handed-over shifts "Do not count as absence" (default) or "Count as absence", and minimum wage by year: 2025 10030, 2026 10320.

Table 21 maps every use case to the screen that realises it. UC-11 has no screen: the System Clock runs the expiry check whenever a screen is opened. Notifications (FR-21) are reached from the bell in the header of every screen.

Table 21. Use cases and screens

Screen	Role	Use cases	Figure
Sign-in (demo tour and accounts)	All	UC-13	8
Home	Worker	UC-05 (accept or decline), UC-04 (entry)	12
Home	Owner	UC-06 (approve or reject)	14
Schedule	Owner, Worker	UC-03, UC-04 (entry), UC-06 (result)	9, 10, 15, 17
Swaps and the 3-step request flow	Worker	UC-04, UC-05, UC-12	11, 13, 18
Me	Worker	UC-02, UC-07 (recording)	16
Staff	Owner	UC-01	21
Pay, Weekly	Owner	UC-08	19
Pay, Monthly	Owner	UC-09	20
Settings	Owner	UC-10	22
Demo tools	All	Demo clock and data reset (drives UC-11)	–
(no screen; runs on every screen change)	System Clock	UC-11	–

9. Requirements validation

Requirements validation checks that the requirements define the system that the customer really wants (Chapter 8). We check the five properties of Chapter 8 with the three techniques it names: requirements reviews, prototyping and test-case generation.

9.1 Requirements review

Two kinds of review are used. First, an internal review: the requirements lead's FR, NFR and business-rule tables are reviewed by the other two members against the problem statement and the scenarios. Second, a review with the owner of the reference store, in which the use cases are read out as scenarios in the owner's own terms and the owner is asked whether each step matches what they need. The owner's answers to interview question 6 (Section 3) decide the default of the BR-06 policy for the reference store; the system keeps both options in any case.

Table 22. Validation checklist (Chapter 8)

Property	Question	How we check it
Validity	Does the system provide the functions that best support the owner's and workers' needs?	Each problem P1–P3 is traced to goals and FRs (P1 → FR-06–FR-10, FR-21; P2 → FR-11, FR-12, FR-14; P3 → FR-15, FR-16). Review with the owner.

Property	Question	How we check it
Consistency	Are there conflicting requirements?	All documents use one specification as single source of truth. State diagrams are checked against the scenarios (Section 7.3).
Completeness	Are all required functions included?	Every FR maps to at least one UC, every UC to at least one screen (Table 21), and every BR to at least one test group (Table 23).
Realism	Can the requirements be implemented with the available time and technology?	MoSCoW priorities; Must requirements form increment 1. Won't list fixed in the proposal. Client-only architecture needs no server budget.
Verifiability	Can each requirement be checked?	Each NFR has a verification method (Table 19). Each FR is stated with a single observable outcome; business rules have numeric boundaries (15 h, 40 h, 5 employees, 90 %, 3 months, 365 days).

9.2 Prototyping

The screens of Section 8 form a working prototype on the demo data. The prototype is used in two ways. The team walks through scenarios S1 to S7 on it and records each step where the screen does not match the use-case description. The owner and the two co-workers are each asked to perform one task: the owner approves the request of scenario S3, one co-worker creates the request of S1, and the other accepts it from *Home* as in S2. We observe whether each task can be completed without help and whether the approval card is understood. The results will be reported in the final report together with any requirement changed because of them.

9.3 Test-case generation

Chapter 8 recommends writing tests for requirements as a way to find requirements that are ambiguous or cannot be tested. We derived one test group per business rule before implementation. The test groups make three boundaries of the requirements explicit: the 15-hour boundary is inclusive (tested at 14.99 h and 15 h), the holiday hours are capped at 40 contractual hours, and shifts ending after midnight are counted with the end on the next day.

Table 23. Business rules and test groups

Business rule	Test group	What the group checks	Related NFR
BR-01	TC-01x	Overlap of time ranges and candidate eligibility	–
BR-02	TC-05x	First-acceptance concurrency: of two acceptances of one request, only the first succeeds	NFR-04
BR-03	TC-02x	Request validity (own shift, future, one open request, deadline)	–
BR-04	TC-06x	Approval effect on the shift and the request	–

Business rule	Test group	What the group checks	Related NFR
BR-05, BR-06, BR-07	TC-08x	Holiday allowance at 14.99 h and 15 h, 40 h cap, both substitution policies, and the worked example (Choi Minho 3.2 h, KRW 33,024)	NFR-10
BR-08	TC-09x	Premium zero at 4 regular employees and non-zero at 5; night hours, including a shift that ends after midnight	NFR-10
BR-09, BR-10	TC-10x	Minimum-wage warning and each probation condition	NFR-10
BR-11	TC-11x	Monthly payroll totals	–
BR-12	TC-12x	Expiry at deadline and at shift start	–

The test groups are written as automated unit tests of the rule functions. Test results will be reported in the final report.

10. Progress and next steps

Work package 3 of the WBS (requirements: elicitation and specification) is complete with this report, and the analysis diagrams of work package 4.1 are drawn. Work package 4.2 (UI design) has produced the screens of Section 8. Requirements validation continues with the review and prototype sessions of Section 9.

The next steps follow the schedule of the proposal. By 2026-10-21 (Week 8) the team completes the design models of work package 4.1: the design class diagram and sequence diagrams SD-1 (request substitute), SD-2 (accept with a concurrent second acceptance), SD-3 (approve with weekly recomputation) and SD-4 (generate monthly payroll). The UI design (4.2) is finished by 2026-10-30, and prompt engineering (5.1) follows in Week 9. Increment 1 (Must use cases) is due at the end of Week 10 (2026-11-06) and increment 2 (Should use cases) at the end of Week 11 (2026-11-13). The system is deployed to GitHub Pages in Week 12 (2026-11-16 to 11-20). The final report, slides and presentation are due in Week 14 (2026-12-04).

References

1. Ministry of Employment and Labor, “2026 minimum wage KRW 10,320 per hour,” MOEL notice, Aug. 2025. https://www.moel.go.kr/news/enews/report/enewsView.do?news_seq=18144
2. Korea Policy Briefing, “2026년 시간당 최저임금 10,320원.” <https://www.korea.kr/news/policyNewsView.do?newsId=148956812>
3. Labor Standards Act (근로기준법), Art. 11, 18, 55, 56; Enforcement Decree Art. 30. <https://www.law.go.kr>
4. Minimum Wage Act (최저임금법), Art. 5 ②; Enforcement Decree Art. 3. <https://www.law.go.kr>
5. Easy-Law (찾기쉬운 생활법령정보), part-time workers' rest and holidays. <https://easylaw.go.kr/CSP/CnpClsMain.laf?popMenu=ov&csmSeq=896&ccfNo=2&cciNo=3&cnpClsNo=1>
6. D. A. Gustafson, *Schaum's Outline of Software Engineering*, McGraw-Hill, 2002.
7. I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

8. OMG, *Unified Modeling Language Specification*, v2.5.1, 2017.
9. sql.js — SQLite compiled to WebAssembly. <https://sql.js.org>
10. Shift-scheduling products examined: Shiftee; 알바몬 근무관리; 알바천국 스케줄; When I Work; Homebase; Deputy.